

GRUPO I 1) static double[] mediaColunas (int[][] m)

O que pedem: devolver um vetor com uma média por coluna.

Ex.: { {5,3}, {15,2}, {10,1} } → col0 média = (5+15+10)/3 = 10 ; col1 média = (3+2+1)/3 = 2 → {10,2}.

Passos

1. Nº de colunas = `m[0].length`
2. Criar `double[] res` com esse tamanho
3. Para cada coluna `c`:
 - somar `m[l][c]` para todas as linhas `l`
 - dividir pelo nº de linhas `m.length`
4. Guardar em `res[c]`

```
static double[] mediaColunas(int[][] m) {
    int linhas = m.length;
    int colunas = m[0].length;

    double[] res = new double[colunas];

    for (int c = 0; c < colunas; c++) {
        int soma = 0;
        for (int l = 0; l < linhas; l++) {
            soma += m[l][c];
        }
        res[c] = (double) soma / linhas;
    }
    return res;
}
```

2) Classe Pessoa (nome + vários micróbios)

O enunciado diz que já existem `Microbio`, `MicrobioTipo` e `Par<P,S>`. Pessoa tem:

- `String nome` e `List<Microbio> microbios`
- `adicionaMicrobio(m)` → `microbios.add(m)`
- `temMicrobio(m)` → `microbios.contains(m)` (o `Microbio` já tem equals)
- `todosMicrobios()` → para cada microbio, criar `Par(nome, tipo)`
- `toString()` → imprimir “Nome: ...” e depois “Microbios:” e listar cada microbio com `microbio.toString()`

```
public class Pessoa {
    private String nome;
    private List<Microbio> microbios;

    public Pessoa(String nome) {
        this.nome = nome;
        this.microbios = new ArrayList<>();
    }

    public void adicionaMicrobio(Microbio m) {
        // assume m != null como diz o enunciado
        microbios.add(m);
    }

    public boolean temMicrobio(Microbio m) {
        return microbios.contains(m);
    }

    public List<Par<String, MicrobioTipo>> todosMicrobios() {
        List<Par<String, MicrobioTipo>> res = new ArrayList<>();
        for (Microbio m : microbios) {
            res.add(new Par<>(m.nome(), m.tipo()));
        }
        return res;
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append("Nome: ").append(nome).append("\n");
        sb.append("Microbios: ").append("\n");
        for (Microbio m : microbios) {
            sb.append(m.toString()).append("\n");
        }
        return sb.toString();
    }
}
```

GRUPO II (ProdutoPedestre, Trekking, Travessia)

A) ProdutoPedestre (abstrata)

1. Guardar **origem** e **destino**
2. Métodos concretos: **origem()**, **destino()**
3. Método template **preco() = distancia() * precoMedioKm()**
4. **toString()** com origem, destino, distância e preço
5. Métodos abstratos: **distancia()** e **precoMedioKm()**

```
public abstract class ProdutoPedestre {
    private String origem;
    private String destino;

    public ProdutoPedestre(String origem, String destino) {
        this.origem = origem;
        this.destino = destino;
    }

    public String origem() { return origem; }
    public String destino() { return destino; }

    public double preco() {
        return distancia() * precoMedioKm();
    }

    @Override
    public String toString() {
        return "De: " + origem() + " ate' " + destino()
            + " Distancia: " + distancia()
            + " Preço:" + preco();
    }

    public abstract double distancia();
    public abstract double precoMedioKm();
}
```

B) Trekking

É um produto simples: tem distância e preço médio por km.

```
public class Trekking extends ProdutoPedestre {
    private double dist;
    private double pmk;

    public Trekking(String origem, String destino, double distancia, double precoMedioKm) {
        super(origem, destino);
        this.dist = distancia;
        this.pmk = precoMedioKm;
    }

    @Override public double distancia() { return dist; }
    @Override public double precoMedioKm() { return pmk; }
}
```

C) Travessia

É composta por vários **ProdutoPedestre** seguidos.

Regras principais:

- começa com 1 produto ("primeiro")
- **adicionaTrekking(proximo)** só se **this.destino().equals(proximo.origem())**
- redefine:
 - **destino()** → destino do último
 - **preco()** → soma dos preços
 - **distancia()** → soma das distâncias
 - **precoMedioKm()** → **preco()/distancia()**
 - **toString()** → além da linha da travessia, imprimir a lista dos produtos que a compõem

```
public class Travessia extends ProdutoPedestre {
    private List<ProdutoPedestre> partes;

    public Travessia(ProdutoPedestre primeiro) {
        super(primeiro.origem(), primeiro.destino()); // destino vai ser redefinido
        this.partes = new ArrayList<>();
        this.partes.add(primeiro);
    }
}
```

```

public void adicionaTrekking(ProdutoPedestre proximo) {
    // pré-condição do enunciado
    if (!this.destino().equals(proximo.origem())) return;
    partes.add(proximo);
}

@Override
public String destino() {
    return partes.get(partes.size() - 1).destino();
}

@Override
public double preco() {
    double soma = 0;
    for (ProdutoPedestre p : partes) soma += p.preco();
    return soma;
}

@Override
public double distancia() {
    double soma = 0;
    for (ProdutoPedestre p : partes) soma += p.distancia();
    return soma;
}

@Override
public double precoMedioKm() {
    return preco() / distancia();
}

@Override
public String toString() {
    StringBuilder sb = new StringBuilder();
    sb.append(super.toString()).append("\n");
    sb.append("Produtos que compoem a travessia: ").append("\n");
    for (ProdutoPedestre p : partes) {
        sb.append(p.toString()).append("\n");
    }
    return sb.toString();
}
}

```

D) equals e que outro método?

Quando redefinimos **equals**, o método que também deve ser redefinido é **hashCode()**.

GRUPO III - 2) totalDistancias

O que pedem: somar **distancia()** de todos os produtos de uma coleção.

1. Receber uma coleção de produtos pedestres (pode ser **Collection<? extends ProdutoPedestre>**)
2. Criar **double soma = 0**
3. Para cada elemento **p**, fazer **soma += p.distancia()**
4. devolver **soma**

```

static double totalDistancias(Collection<? extends ProdutoPedestre> col) {
    double soma = 0;
    for (ProdutoPedestre p : col) {
        soma += p.distancia();
    }
    return soma;
}

```